

Efficient Emulation of Smooth Functions for Bayesian Inference

Oleh Savchuk, APS DNP Chicago, 20th October 2025



**Facility for
Rare Isotope Beams**
at Michigan State University

Bayesian statistics

Bayes theorem for posterior:

The diagram illustrates Bayes' theorem for the posterior probability. It shows the equation $P(\theta | X) = \frac{P(X|\theta) P(\theta)}{P(X)}$. The terms are represented by colored boxes: $P(\theta | X)$ is in a blue box, $P(X|\theta)$ is in a teal box, $P(\theta)$ is in a blue box, and $P(X)$ is in a blue box. Arrows point from the text below to each of these boxes: from 'Parameters θ ' to the θ box, from 'observations X ' to the X box, from 'observations given parameters' to the $P(X|\theta)$ box, from 'prior' to the $P(\theta)$ box, and from 'probability of observation' to the $P(X)$ box.

$$P(\theta | X) = \frac{P(X|\theta) P(\theta)}{P(X)}$$

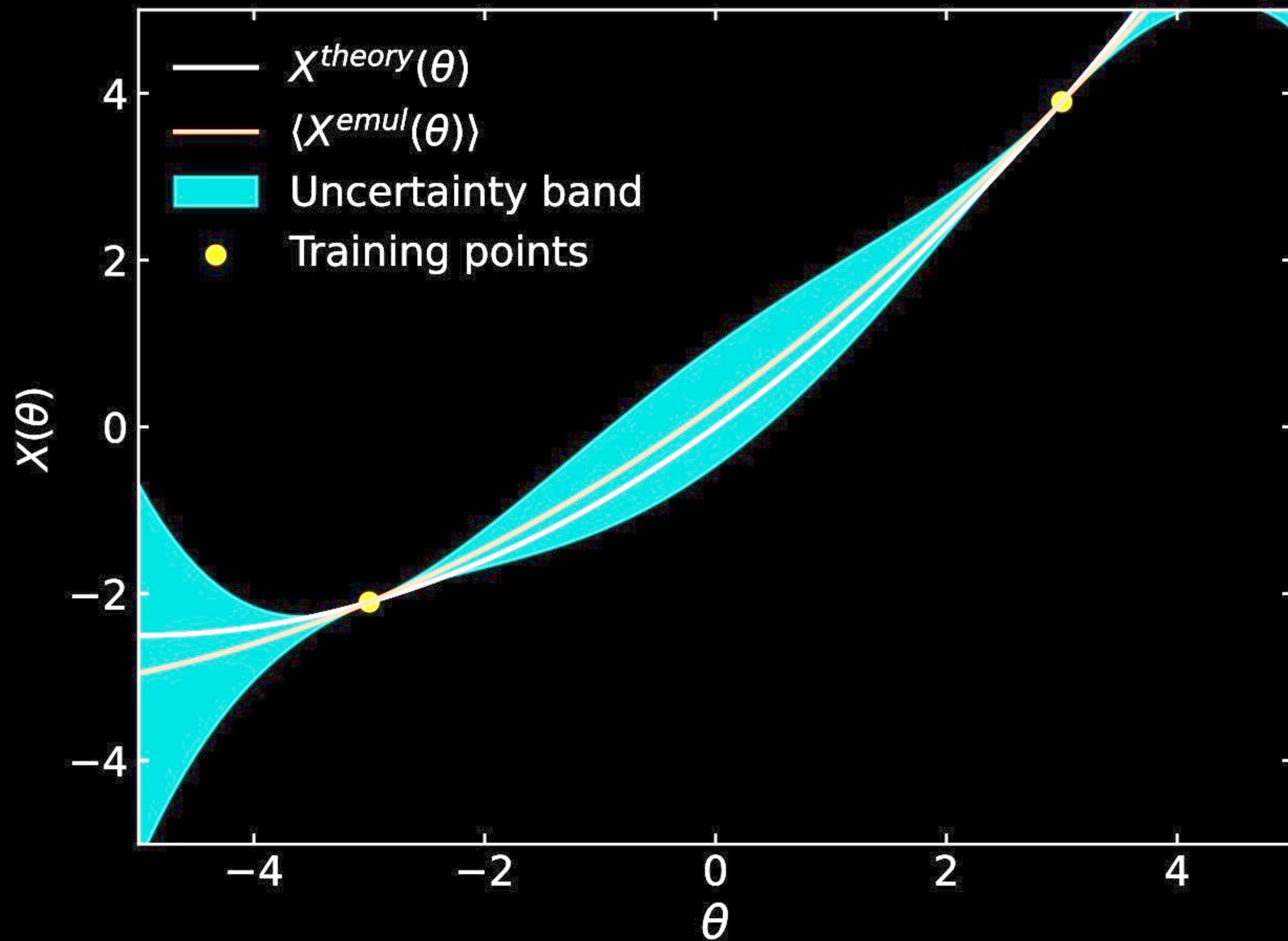
Parameters θ , observations X , observations given parameters, prior, probability of observation

$$P(X | \theta) = \exp \left\{ -\frac{1}{2} \sum \frac{(X_i - X_i^{theory}(\theta))^2}{\sigma_i^2} - \sum \ln \sqrt{2\pi\sigma_i^2} \right\}$$

Might require enormous number of model runs at different parameters θ - computationally expensive

Emulators

Purpose: replace full-model (expensive) with emulators (cheap) trained on much smaller number of full-model runs



Smooth emulator

Taylor expansion with only a few first terms - exemplary smooth function

One might try to think about an emulator in the same way - Taylor expansion with coefficients that can be learned from data with some uncertainty:

$$X^{emulator}(\theta) = \sum_i A_i T_i(\theta) = \sum_{\{n\}} A_{\{n\}} \sqrt{\frac{(n_1 + \dots + n_N)!}{n_1! \dots n_N!}} \prod_i \left(\frac{\theta_i}{\Lambda} \right)^{n_i}$$

With coefficients following next distributions:

$$P(\vec{A}) \sim \delta(A_i T_i(\theta_j) - X^{theory}(\theta_j)) \exp \left\{ -\frac{A_i^2}{2\sigma_A^2} \right\} \quad \text{- ensures emulator matches model at training points, leaves room for fluctuations elsewhere}$$

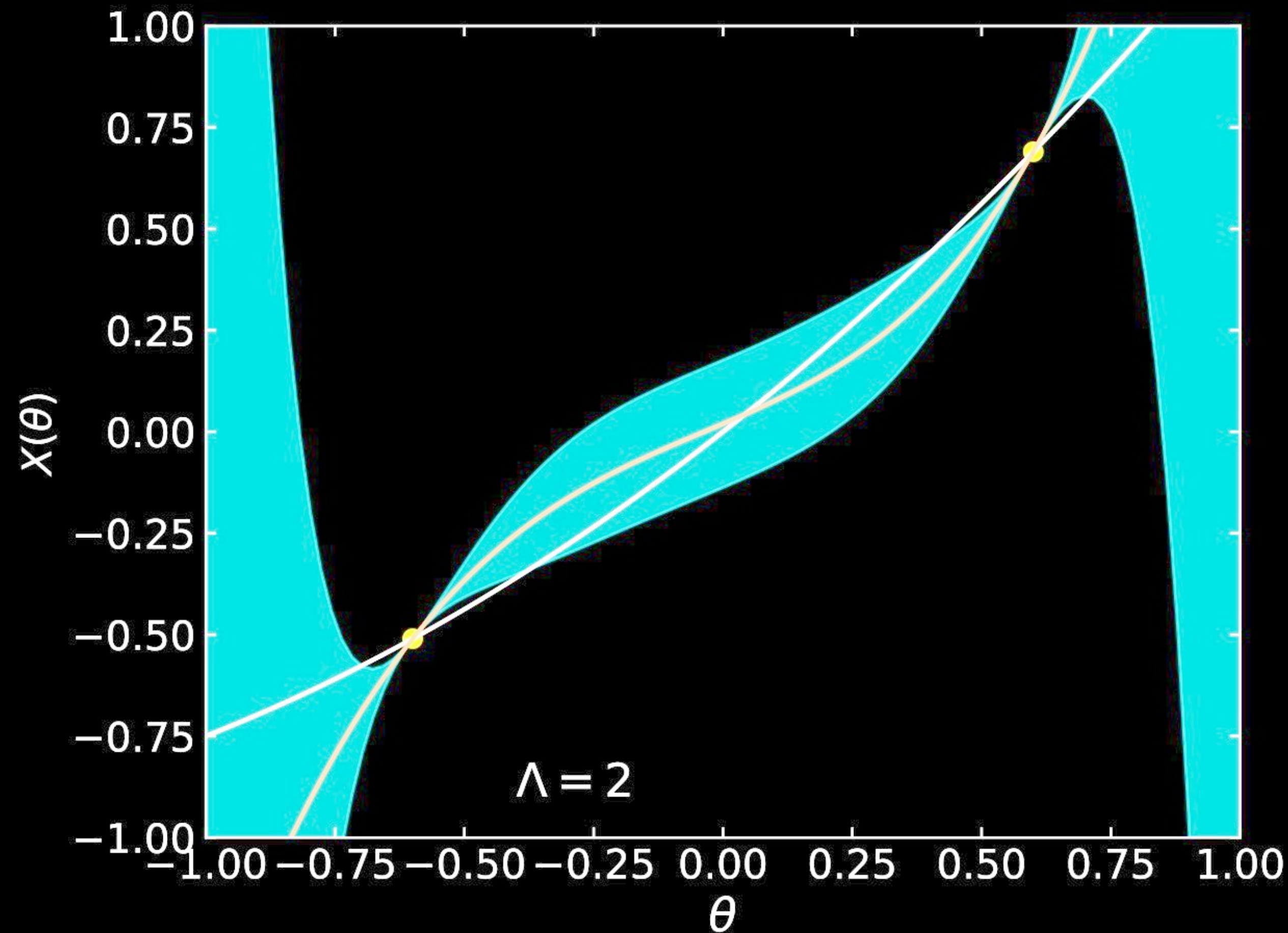
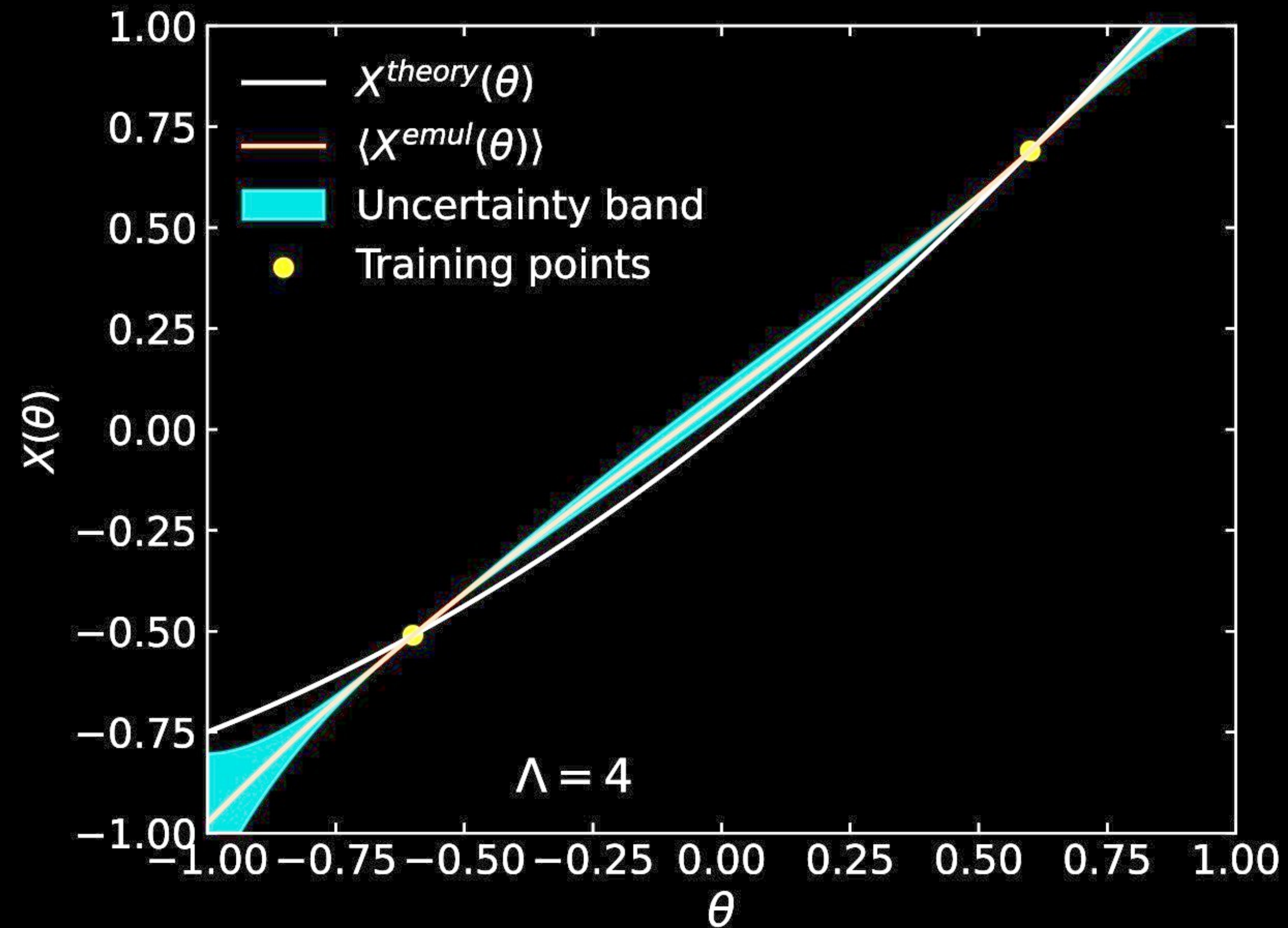
Theoretical model uncertainty can be included:

$$\delta(X_j^{th} - X_j^{emu}) \rightarrow \frac{|\Delta|}{(\sqrt{2\pi})^{N_{train}}} \exp \left\{ -\frac{1}{2} (X_j^{th} - X_j^{emu}) \Delta_{jk}^{-1} (X_k^{th} - X_k^{emu}) \right\}$$

If all terms are included equivalent to the GP on the level of variance

Λ controls smoothness

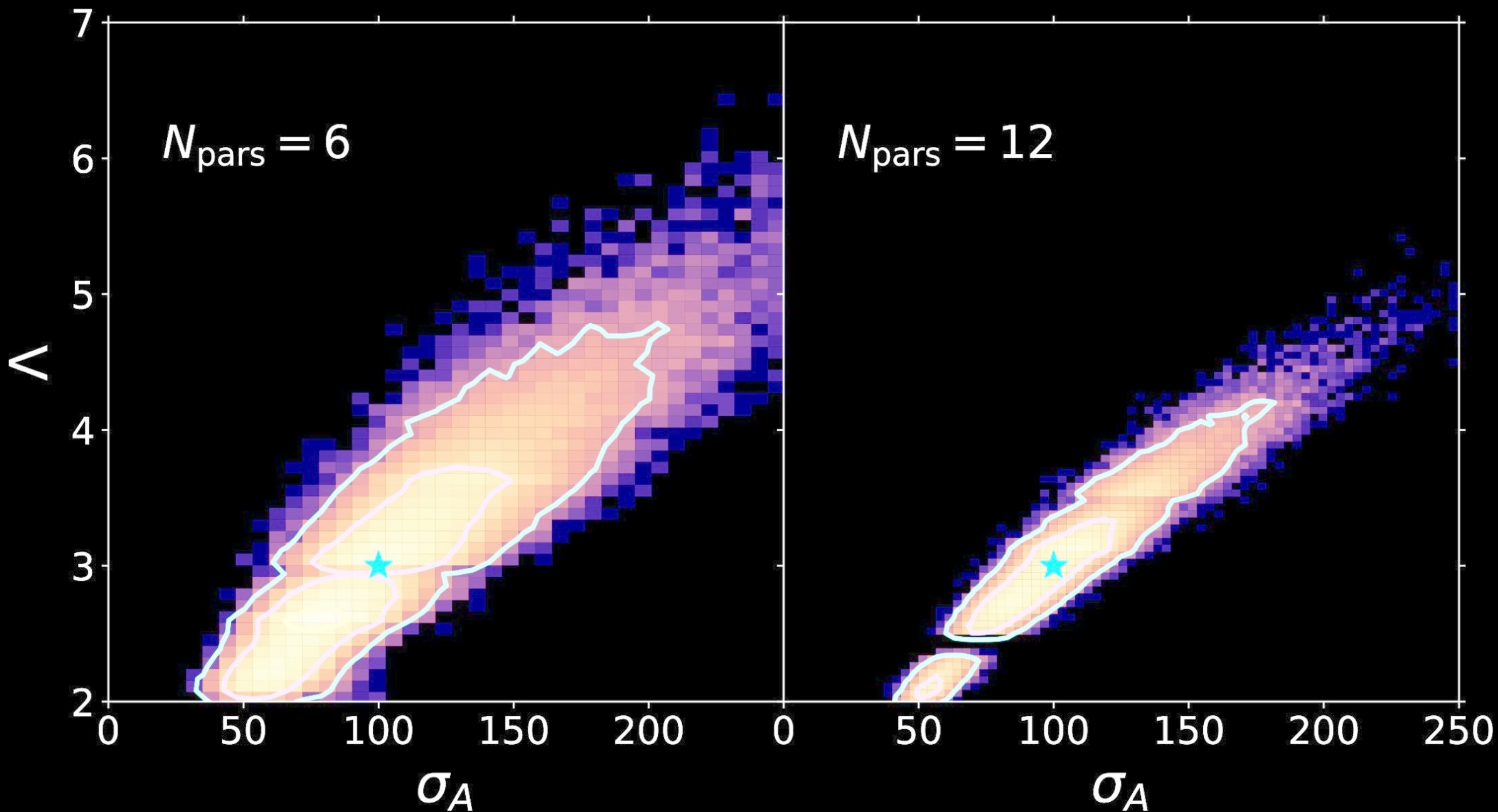
$$X^{emulator}(\theta) = \sum_i A_i T_i(\theta) = \sum_{\{n\}} A_{\{n\}} \sqrt{\frac{(n_1 + \dots + n_N)!}{n_1! \dots n_N!}} \prod_i \left(\frac{\theta_i}{\Lambda}\right)^{n_i}$$



Parameters

σ , Λ can be obtained from maximal likelihood

$$P(X^{th} | \sigma_A, \Lambda) = \int dA_i P(A) = \frac{\sigma_A^{N_{train}} \exp \left\{ -\frac{1}{2\sigma_A^2} X_j^{th} \left(O_{jk} + \frac{\Delta_{jk}}{\sigma_A^2} \right) X_k^{th} \right\}}{\sqrt{\det \left(O_{jk} + \frac{\Delta_{jk}}{\sigma_A^2} \right)}} \quad O_{lm} = T_l(\theta_l) T_l(\theta_m)$$

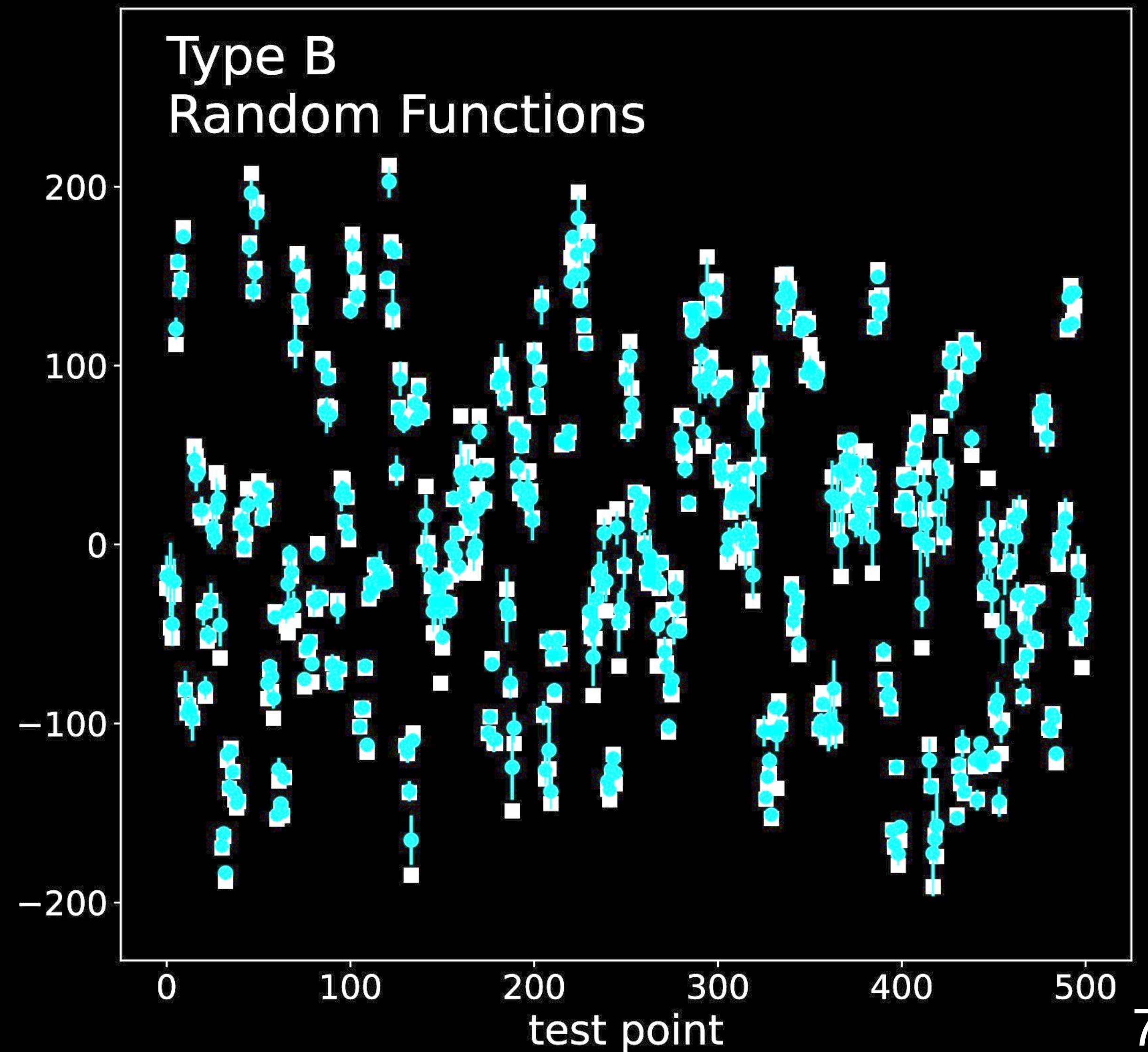
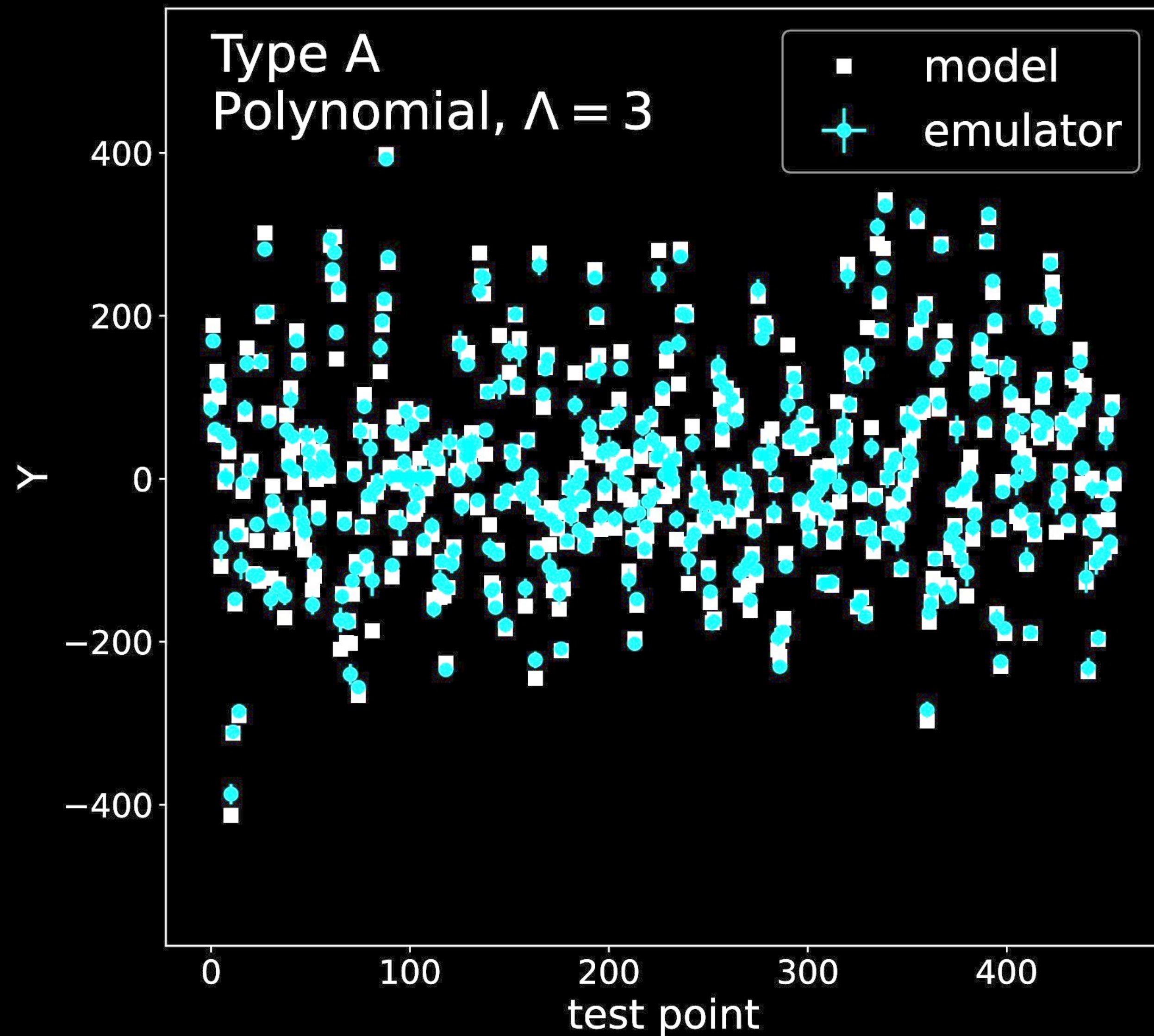


Subtle correlation between width of coefficient's prior and smoothness is observed

Leads to small variations of uncertainty

Emulator Uncertainty

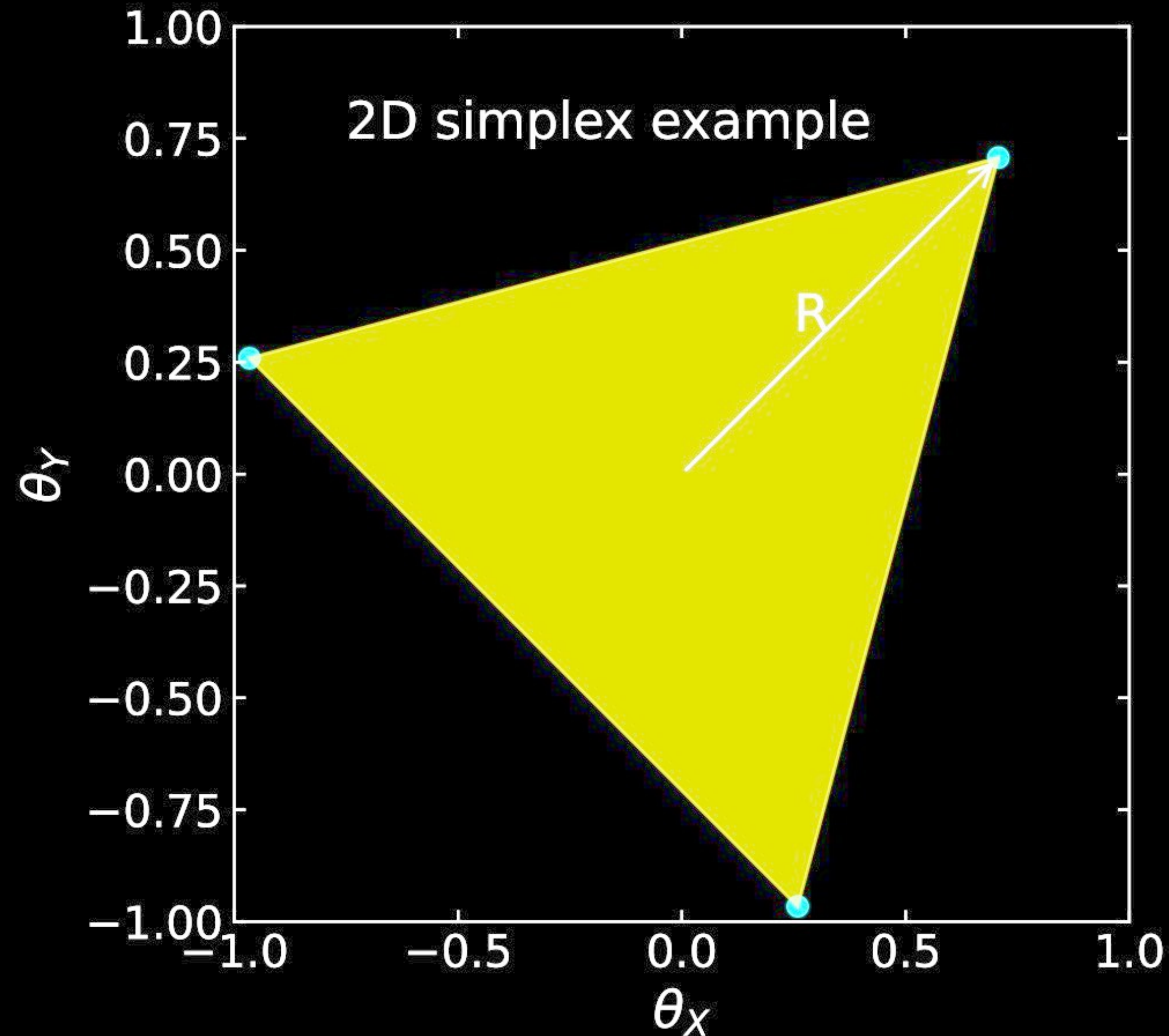
$$\sigma^2(\theta) = \langle \Delta X^{em}(\theta) \Delta X^{em}(\theta) \rangle = \sigma_A^2 (T_i(\theta) T_i(\theta) - (O_{lm} + \sigma_A^{-2} \Delta_{lm})^{-1} S_l(\theta) S_m(\theta)) \quad S_l(\theta) = T_i(\theta_l) T_i(\theta)$$



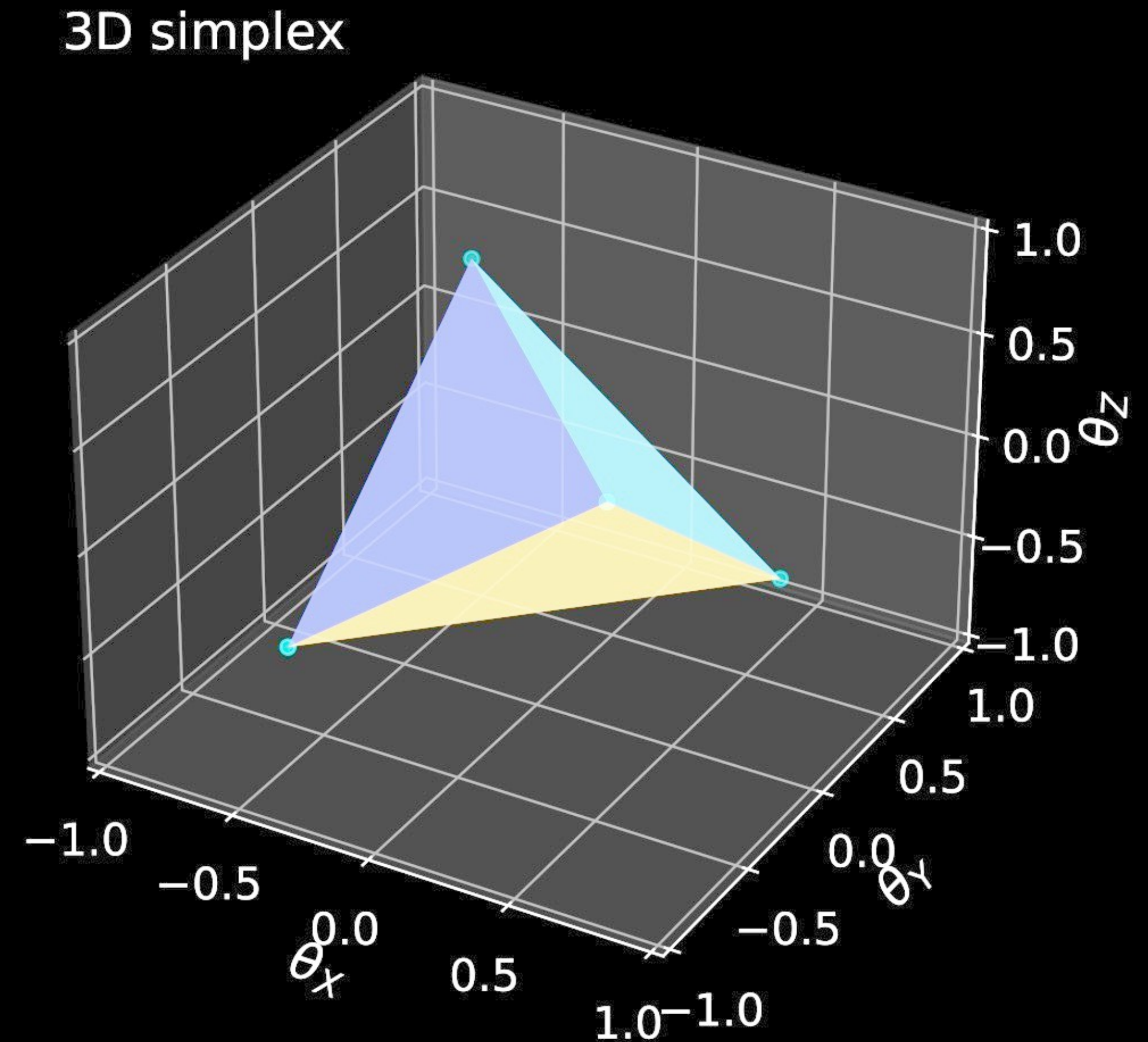
Training point optimization

Starting from linear approximation in Taylor series - geometric location of training points with most symmetry - Simplex

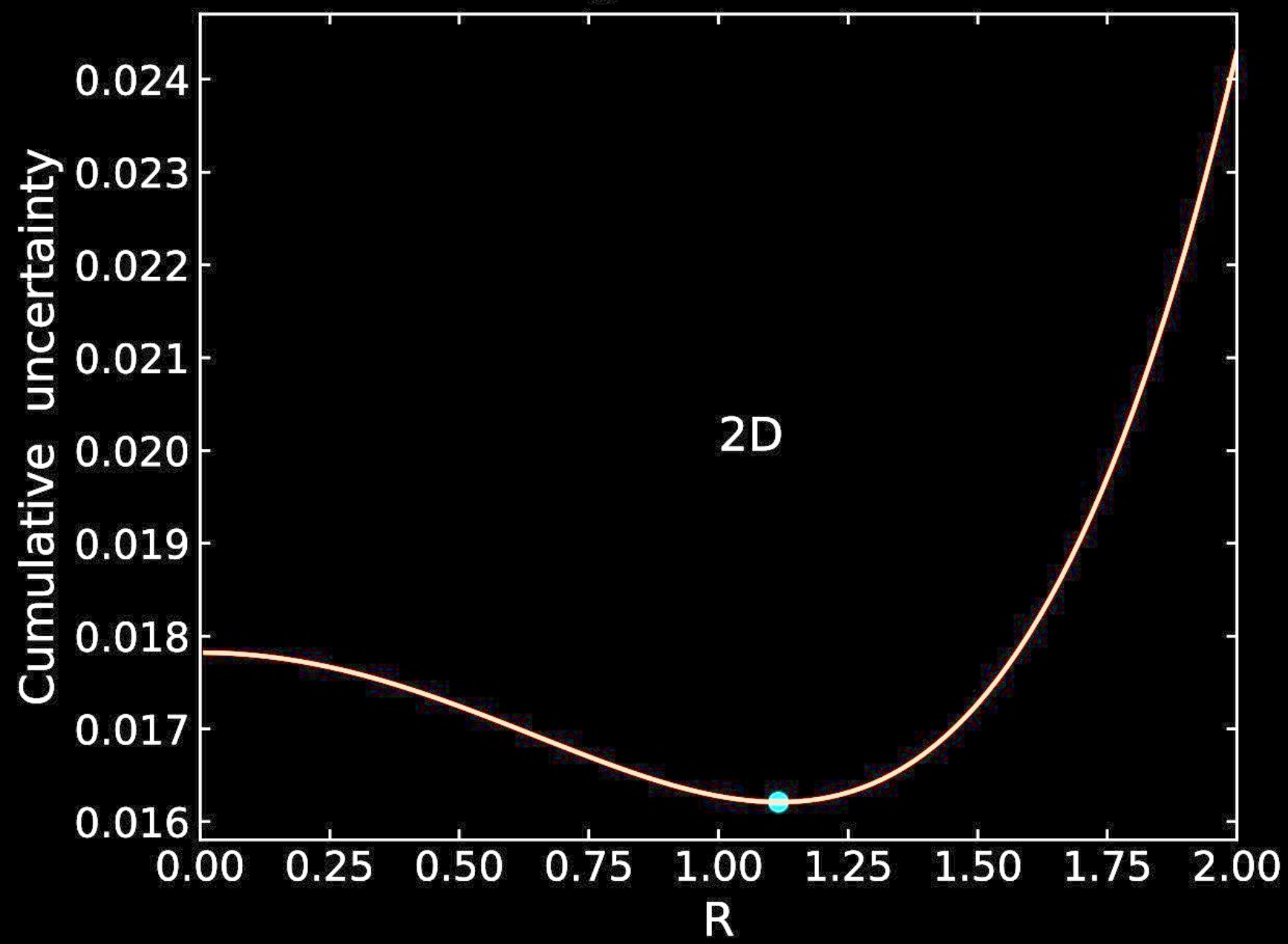
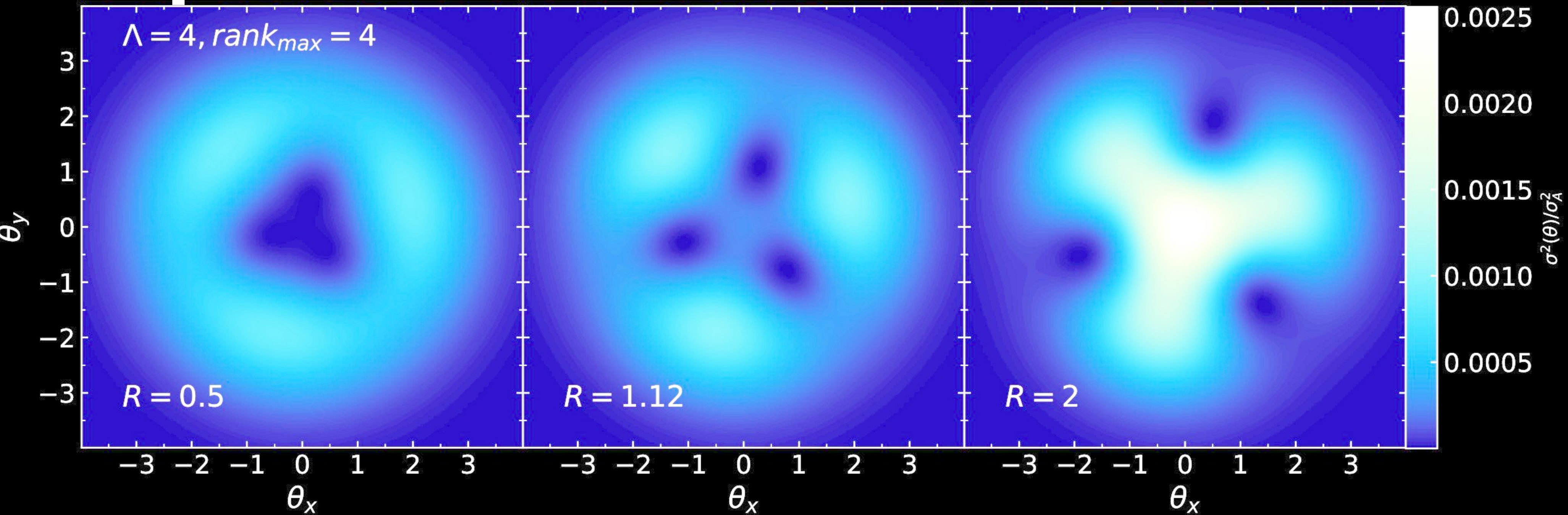
Linear - $N_{train} = N_{param} + 1$



Quadratic $N_{train} = (N_{param} + 1)N_{param}/2$



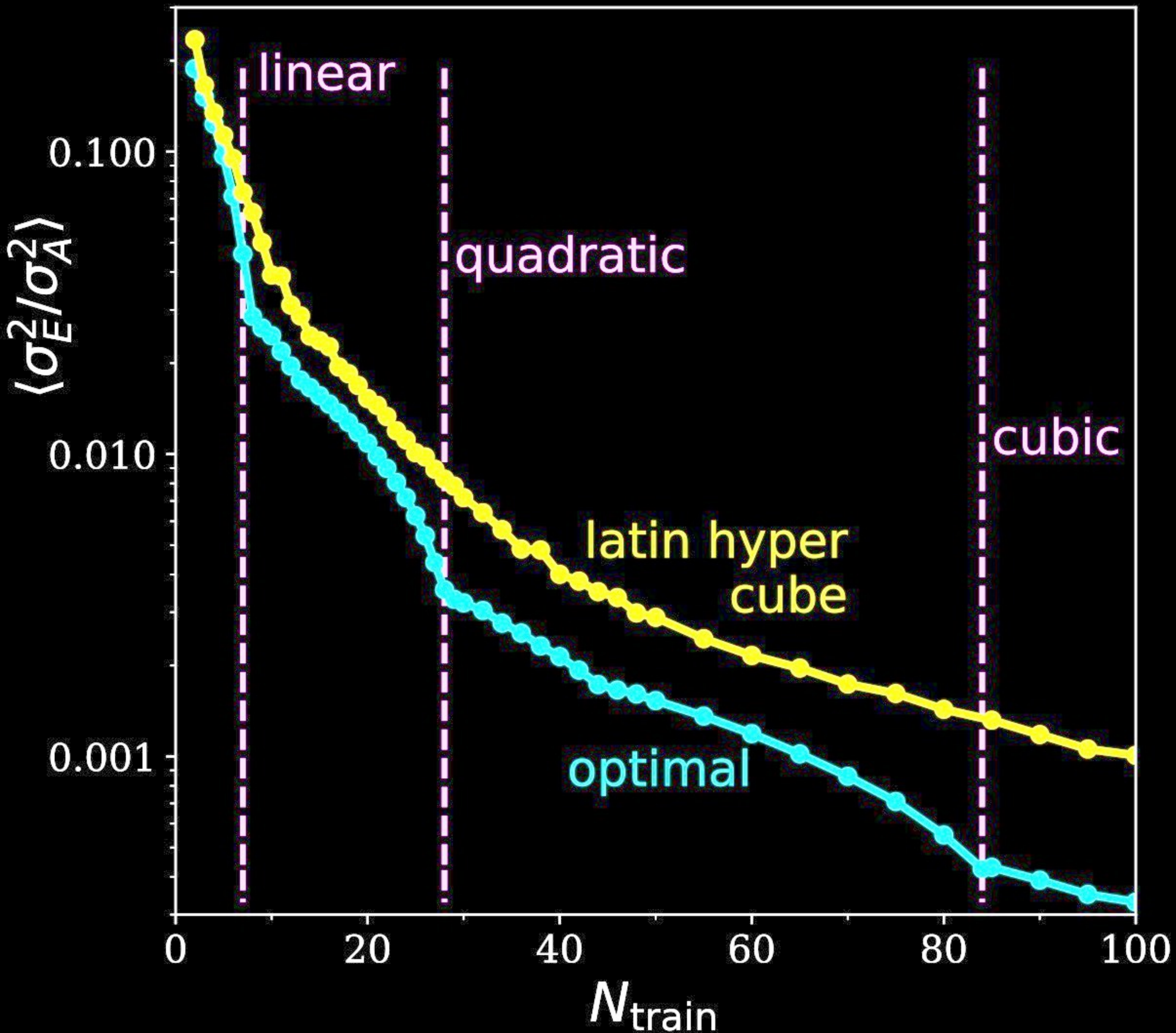
Simplex case



$$CU = \int d\theta \frac{\sigma^2(\theta)}{(2\pi)^{N/2} \sigma_A^2} \exp \left\{ -\frac{\theta^2}{2} \right\}$$

Uncertainty has a minimum - there is a difference in training point selection

Conclusions



- Smooth emulator is fast and easy to train
- Reduces to Gaussian Process emulators but is more tractable with regards to optimization
- Does good job at describing uncertainty
- Written in C++, works in Python
- Training point choice matters - a route for improving emulator performance